

Project Overview

- **Build an ASP.NET Core 10 MVC news site web application and deploy to Azure App Service.**
- Support user registration/login (email confirmation), subscriptions, article management, role-based (or policies-based) access, external data APIs and Azure Serverless features (Functions).
- Use SQL Database on Lexicon Server. Azure Blob Storage for images.
- Duration: 8–9 weeks (deadline for Group 26-20: November 5).

Project goals

- Feature-complete news site supporting readers and staff (Writer, Editor, Admin).
- Clean architecture (services, repositories, EF Core, dependency injection).
- Production-ready deployment on Azure, with automated tasks via Azure Functions.
- Basic analytics and historical data storage and visualizations.

Core pages & features (minimum)

1. Front page
 - Sections: Latest, Editor's Choice, Most Popular.
 - Optionally personalized recommendations for logged-in users.
 -
2. News sections (minimum)
 - Local, Sweden, World, Weather, Economy, Sport.
3. Article detail page
 - Full article, image, metadata (date, category), likes/views.
4. Authentication & account
 - Register and login (email confirmation).
 - "My Page" for profile, reset password, view/edit subscriptions, sign up/edit personalized newsletter.

5. Subscriptions

- “Subscribe now” flow (shows only to users without active subscription).
- Form validates credit card fields (dummy validation acceptable; optional: integrate Stripe).
- Generate subscription confirmation email.

6. Privacy & cookies

- Cookie consent banner and privacy page.

7. Admin pages

- Add/edit articles (Writer or Journalist role), including image upload.
- Mark Editor’s choice.
- View subscription statistics and user/role management (add employees, assign roles).

8. Search

- Search function for current news (and optionally archived news).

9. Media

- Use Azure Blob Storage for article images; resize images via Azure Function blob trigger.

External API integrations (minimum)

- Weather API (required).
- At least one other API (examples): currency rates, electricity prices, business data.
- Save historical data into Azure Table Storage for visualization.

Azure Functions (tasks)

- Send personalized newsletter (e.g., weekly).
- Archive news automatically (e.g., after 30 days).
- Send email notifications when subscriptions near expiry.
- Resize images on upload (Blob trigger).
- Save external API historical data to Azure Tables.

Technical stack & architecture

- ASP.NET Core 10 MVC.
- Authentication: ASP.NET Core Identity (Individual Accounts).
- Database: SQL Database on Lexicon Server.
- Services: Implement service layer for business logic (e.g., IArticleService).
- Repositories: Implement repository layer for CRUD (e.g., IArticleRepository). (Optional)
- Blob Storage: Azure Blob Containers for images.
- Azure Functions: serverless background tasks with triggers.
- SMTP Service for emails.
- CI/CD: GitHub Actions (or Azure DevOps) to build and deploy to App Service.
- Optional: Use Azure Key Vault for secrets.
- Optional: Stripe for payments;

Suggested data model (core classes/properties)

- User (extends IdentityUser)
 - FirstName, LastName, DOB, Subscriptions
- Article
 - Id, DateStamp, Slug (LinkText), Headline, ContentSummary, Content, Views, Likes, ImageUrl, CategoryId, IsArchived
- Category
 - Id, Name
- Subscription
 - Id, SubscriptionTypeId, Price (historical), Created, Expires, UserId, PaymentComplete
- SubscriptionType
 - Id, TypeName, Description, Price
- WeatherForecast (DTO or View model for API responses)
 - Id, Summary, TemperatureC, TemperatureF, Humidity, WindSpeed, Date, City, Description, Icon
- HistoricalData (for other APIs)
 - Properties depending on Json response

UI components & patterns

- Use a View Component for reusable pieces (e.g., LatestArticles, EditorChoice, TopCategories).
- Razor Views for pages and partials.
- Razor Pages for Identity (Register, Login, etc.)
- Validation: client-side (Unobtrusive/JS) + server-side (DataAnnotations or custom validators).
- Responsive design (Bootstrap or similar).

Roles & authorization

- Roles: Reader (default), Writer, Editor, Admin.
- Writers: create drafts; Editors: publish and mark Editor's Choice; Admins: manage users, roles and site settings.
- Use role-based/policy-based authorization attributes on controllers/actions.

Search & analytics

- Basic DB-backed search (Title, Summary, Content)
- Show subscription graphs (ChartJS, Syncfusion, or other).
- Store historical API data in Azure Tables and present time-series charts.

Testing & quality (Optional)

- Unit tests for services and business logic.
- Integration tests for key flows (register, subscribe, create article).
- Manual QA for cross-browser/responsive behavior.

Security & privacy

- Protect secrets in Key Vault or App Service configuration; do not store credentials in repo.
- GDPR / privacy: cookie consent, allow (soft)deleting accounts, explain data usage in Privacy page.
- Email verification and password reset flows secured with time-limited authentication tokens.

- Optional: Implement input sanitization (prevent XSS) on article content and user input.

Deployment & CI/CD

- GitHub repo with branches and PR-based reviews.
- GitHub Actions pipeline:
 - Build & test on push to assigned branch.
 - Deploy to Azure App Service.
 - Deploy Azure Functions (optional).
- Environment configuration via appsettings.json (Development) and Key Vault/app configuration.

Suggested project modules

Project setup and planning

- Project Management (tools, planning, etc)
- Create repo, solution, projects (Web app, Functions).
- Create basic DB models and EF migrations.
- Setup SQL DB on server (and LocalDb)
- Setup Azure resources skeleton (App Service, Storage account, Function Apps).

Auth, user pages, roles

- Implement Identity, registration, email confirmation, login.
- Build My Page and profile management.
- Implement user and role management (Admin UI).

Articles

- Article CRUD (Writer role), image upload to Blob Storage.
- Front page sections (View Components: Latest, Editor's choice).
- Article Details.

Subscriptions & payments

- Subscription model and UI.
- Dummy credit card validation; optionally integrate Stripe test mode.

- Subscription confirmation email and subscription statistics backend.

APIs & Azure Functions

- Integrate Weather API and one additional API; store historical data in Azure Tables.
- Implement Azure Functions: image resizing, archiving, newsletter.

Search, analytics, polish

- Implement search; charts for subscriptions and historical data.
- Cookie consent, privacy page, accessibility checks.

Finalize, testing & deploy

- Final testing, fix issues, final deploy, automated functions scheduling.
- Create deployment and user documentation (optional).

Deliverables checklist (minimum)

- Running web app deployed to Azure App Service (URL).
- SQL DB with sample data (on Lexicon Server).
- Azure Blob Storage for images in use.
- At least one Azure Function implemented.
- User registration/login with email confirmation.
- Subscription workflow with email confirmation.
- Admin pages for article creation, user and role management.
- Weather API integration and one other external API, with history storage (Azure Tables).
- README (on GitHub) with setup instructions and credentials for test users.

Acceptance criteria

- Functionality: Register/login, confirm email, create subscription, admin article CRUD, page navigation works.
- Persistence: Articles, users, subscriptions stored in SQL DB; images in Blob Storage.
- Azure: App Service, Function App, Blob, Azure Tables used as specified.
- Security: HTTPS enabled, password reset and email verification work, role-based access enforced.

- UX: Responsive layout, cookie consent present.

Optional goals

- Integrate Stripe for live payment handling (test mode).
- Add AI summarization generation using Azure OpenAI for article.
- Add text-to-speech (TTS) generation using Azure AI for article content.
- Implement push notifications or real-time updates (SignalR).
- Implement tests for services layer.

Implementation tips and best practices

- Keep controllers thin; put logic in services.
- Use DTOs/ViewModels for views and API responses.
- Use asynchronous EF Core calls (async/await).
- Centralize configuration for API keys and connection strings.
- Use logging (Application Insights in Azure) to monitor errors and performance.

Resources & links you might need

- ASP.NET Core 10 docs (Identity, EF Core).
 - Azure App Service, Function App, Blob Storage, Table Storage docs.
 - Stripe docs for .NET (if you use payments).
 - ChartJS / Syncfusion docs for charts.
 - SMTP Service Provider (e.g. Gmail) docs for emails.
-

Optional – Add a web shop

If you have time and are interested in repeating implementation of a web shop:

Goals

- Offer a simple product catalog, shopping cart, checkout (secure, test-mode payments), order persistence, and basic admin management. Reuse existing infrastructure (Identity, Blob Storage, email service).

Minimum feature set

- Product catalog page and product detail pages (image, description, price, category).
- Shopping cart (session-based or persisted per user) with add/edit/remove functionality.
- Checkout flow using Stripe Checkout in test mode (recommended) or a validated dummy payment form.
- Order and OrderItem records in the SQL .
- Order confirmation email to the buyer.
- Admin UI to add/edit products and view orders.
- Display order history in My Page for logged-in users.

Suggested data model (EF Core entities)

- Product: Id, Name, Description, Price, ImageUrl, Category, optional), IsActive , StockQuantity
- Order: Id, UserId, Created,ShippingAddress (optional fields)
- OrderItem: Id, OrderId, ProductId, UnitPrice
- (Optional) Cart/CartItem if you want persisted carts

Pages / Endpoints to implement

- /Shop (catalog), /Shop/{id} (product details)
- /Cart (view/edit), /Checkout
- /Orders/History (My Page), /Admin/Products (CRUD), /Admin/Orders (view/update status)

Integration & reuse

- Use Azure Blob Storage for product images (same workflow used for article images).
- Use existing Identity for user accounts and My Page order history.
- Use the app's email service (SMTP) for order confirmations.
- Protect admin product/order pages via the existing role-based authorization (e.g. Admin role).